Day 107 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 16, Day 3 reference

Watch: StatQuest: Random Forests Part 1 - Building, Using and Evaluating | 22. Bagging and Random Forests (Inside Bloomberg)

`day107_random_forests.py`, segment by segment, in the order `main()` uses things. Day 105's split search is imported unchanged, and scikit-learn's forest is used only as an outside check. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: mtry = features a question may see · B = trees · p = features · σ^2 = one tree's variance · ρ = correlation between two trees · OOB = out of bag

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day106_pruning as d106
d105 = d106.d105
gini, best_split, stratified_split = d105.gini, d105.best_split, d105.stratified_split
TREES = 100
RUNS, SUB, BAG = 30, 300, 64
COUNTS = [1, 2, 4, 8, 16, 32, 64]
MTRYS = [1, 2, 3, 5, 8, 12, 20, 30]
NOISE = [0, 10, 30, 100, 300]
LEAVES = [1, 2, 5, 10, 20, 40]
SPLITS = {"Titanic": 20, "Breast cancer": 20, "Digits": 6}
```

- `d105 = d106.d105` — Day 106 already imported Day 105, so its module is reused rather than loaded twice.
- `RUNS, SUB, BAG = 30, 300, 64` — tuple assignment: three settings on one line.
- `SPLITS = {"Titanic": 20, ...}` — a dictionary, because digits is slower and gets fewer splits.

Segment 1 — The forest

grow_random: a question sees mtry features

CODE

```
def grow_random(X, y, n_classes, mtry, rng, depth=0, max_depth=None, min_leaf=1):
    counts = np.bincount(y, minlength=n_classes)
    node = {"counts": counts, "pred": int(np.argmax(counts))}
    if (max_depth is not None and depth >= max_depth) or counts.max() == len(y):
        return node
    order = rng.permutation(X.shape[1])
    feats = order[:mtry]; s = best_split(X[:, feats], y, n_classes, gini, min_leaf); k = mtry
    while s is None and k < X.shape[1]:
        feats = order[k:k + 1]; s = best_split(X[:, feats], y, n_classes, gini, min_leaf); k += 1
    if s is None:
        return node
    f, t = int(feats[s[0]]), s[1]; m = X[:, f] <= t
    node.update(feature=f, threshold=t, gain=s[2],
                left=grow_random(X[m], y[m], n_classes, mtry, rng, depth + 1, max_depth,
                    min_leaf),
                right=grow_random(X[~m], y[~m], n_classes, mtry, rng, depth + 1, max_depth,
                    min_leaf))
    return node
```

- `order = rng.permutation(X.shape[1])` — the feature numbers in a random order.
- `X[:, feats]` — fancy indexing: only those columns, in that order.
- `feats = order[k:k + 1]` — a slice of length one, so best_split still receives a two-dimensional table.
- `f, t = int(feats[s[0]]), s[1]` — best_split numbers features within the subset, so its answer is translated back.

proba: many rows through one tree

CODE

```
def proba(node, X, n_classes):
    out = np.zeros((len(X), n_classes))
    stack = [(node, np.arange(len(X)))]
    while stack:
        n, idx = stack.pop()
        if "feature" not in n:
            out[idx] = n["counts"] / n["counts"].sum()
            continue
        m = X[idx, n["feature"]] <= n["threshold"]
        stack.append((n["left"], idx[m])); stack.append((n["right"], idx[~m]))
    return out
```

- `stack = [(node, np.arange(len(X)))]` — a list used as a to-do stack: each entry is a node and the rows that reached it.
- `n, idx = stack.pop()` — takes the last entry; no recursion is needed.
- `idx[m], idx[~m]` — the row numbers that answered yes and no: a whole group moves down at once.

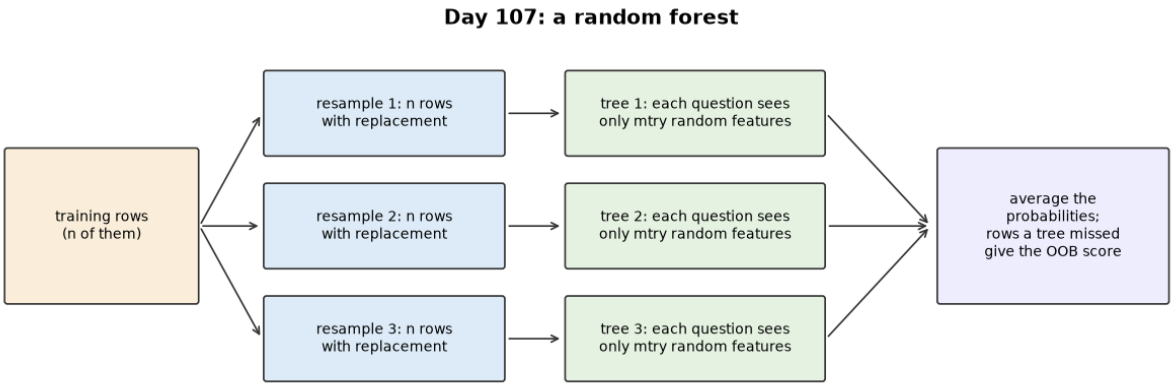
fit_forest, forest_proba and oob_accuracy

CODE

```
def fit_forest(X, y, n_classes, n_trees, mtry, seed, max_depth=None, min_leaf=1):
    rng = np.random.RandomState(seed); n = len(y); trees, bags = [], []
    for _ in range(n_trees):
        idx = rng.randint(0, n, n); bags.append(idx)
        trees.append(grow_random(X[idx], y[idx], n_classes, mtry, rng, max_depth=max_depth,
            min_leaf=min_leaf))
    return trees, bags
def forest_proba(trees, X, n_classes):
    return np.mean([proba(t, X, n_classes) for t in trees], axis=0)
def oob_accuracy(trees, bags, X, y, n_classes):
    votes = np.zeros((len(y), n_classes))
    for t, idx in zip(trees, bags):
        out = np.setdiff1d(np.arange(len(y)), idx)
        votes[out] += proba(t, X[out], n_classes)
    seen = votes.sum(1) > 0
    return float(np.mean(np.argmax(votes[seen], 1) == y[seen])), float(seen.mean())
```

- `rng.randint(0, n, n)` — n positions with replacement: the bootstrap.
- `np.mean([... for t in trees], axis=0)` — stacks the trees' tables, and averages down the first axis.
- `np.setdiff1d(np.arange(len(y)), idx)` — the row numbers missing from idx: this tree's out-of-bag rows.
- `votes[out] += proba(...)` — adds to those rows only, leaving the others alone.

SYSTEM DIAGRAM



A random forest.

Segment 2 — The measurements

InOrder: a stand-in for a random generator

CODE

```
class InOrder:
    def permutation(self, n):
        return np.arange(n)
```

- `class InOrder:` — any object with a permutation method works, so this replaces the generator for one check.
- `return np.arange(n)` — 'random' order that is simply 0, 1, 2 ...: duck typing.

decomposition, and the floor

MATH

$$Var(\text{average of } B \text{ trees}) = \rho\sigma^2 + (1 - \rho)\sigma^2/B$$

CODE

```
def decomposition(P, y):
    mean = P.mean(0)
    return float(((P - mean) ** 2).mean()), float(((mean - y) ** 2).mean())
...
    q = P - P.mean((0, 1), keepdims=True)
    sig = float((q ** 2).mean())
    s = q.sum(1)
    cov = float(((s ** 2 - (q ** 2).sum(1)) / (BAG * (BAG - 1))).mean())
    rho = cov / sig
```

- `P.mean((0, 1), keepdims=True)` — averages over two axes (training set, tree), leaving one mean per test row.
- `s = q.sum(1)` — the total deviation per forest; squaring it gives every pair of trees at once.
- `(s ** 2 - (q ** 2).sum(1)) / (BAG * (BAG - 1))` — removes the tree-with-itself terms, leaving the average over pairs of different trees.

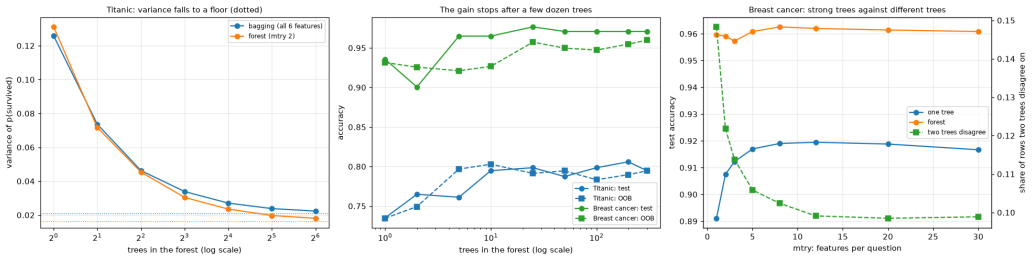
Disagreement between trees, inside demo_mtry

CODE

```
P = np.stack([proba(t, X[te], 2) for t in trees])
lab = P.argmax(2)
one.append(np.mean(lab == y[te]))
dis.append(np.mean(lab[:50] != lab[50:]))
```

- `np.stack([...])` — a three-dimensional array: tree, row, class.
- `P.argmax(2)` — along the class axis: each tree's answer for each row.
- `lab[:50] != lab[50:]` — compares tree 0 with tree 50, tree 1 with tree 51 and so on.

REAL GRAPH



Real graph from the script: variance against forest size, accuracy against trees, and mtry.

Quick Syntax Glossary — new or reinforced constructs

- `rng.permutation(n)` — 0 to n - 1 in random order.
- `np.setdiff1d(a, b)` — values in a that are not in b.
- `np.stack(list)` — joins equal-shaped arrays along a new axis.
- `a.mean((0, 1), keepdims=True)` — averages over two axes, keeping them as size 1.
- `class X: def f(self): ...` — a small object with one method, here to stand in for a generator.

End of Day 107 syntax reference. For the experiments and what they showed, see the Day 107 complete guide.